TKV: Keep What the Basis Cannot Explain. Frozen Low-Rank KV Caches and the Attention-Free, Query-Agnostic Selection Criterion They Provide

David Aror
Independent Research
tejirid.aror@gmail.com

Abstract

When a long context is compressed before the question is known — cached documents, multi-turn chat, prompt reuse — a KV cache must decide which tokens to keep without seeing the queries that will use them. Attention-based selection (H2O, SnapKV) scores tokens by how *past* queries attended to them and requires materialized attention probabilities. We study **TKV**, a KV cache that fits a low-rank basis to pre-RoPE keys during a short warmup and then freezes it, and show that its most useful product is not the compressed storage but a *signal*: each token's projection residual $\|k - B^\top Bk\|$ — how much of the token the basis cannot explain. We first show a limit on what any such rule can promise: no query-agnostic policy that *evicts* tokens can bound its error, because a later query may align with exactly what was dropped. Keeping the non-selected tokens as projections instead (**TKV-R**) admits a bound, and ranking by the residual is what minimizes it — making TKV-R the only configuration in our comparison with a worst-case guarantee. It is also competitive: at matched KV memory it ties the best query-agnostic method on Mistral-7B (0.93 top-1 needle retrieval) and matches the best baseline on TinyLlama-1.1B (0.77 vs. 0.77 H2O, ahead of 0.67 SnapKV and 0.60 StreamingLLM). **TKV-S**, the evicting variant, uses the same residual without attention scores or knowledge of the question. The bound holds because RoPE is a rotation; its evicting counterpart cannot hold at all, which places TKV-S, H2O, SnapKV and KVzip alike in the heuristic regime. It beats H2O on all three models tested (significantly on Mistral-7B and Qwen2.5-14B), matches SnapKV on Mistral-7B, beats it on TinyLlama-1.1B (0.93 vs. 0.67), and trails it on Qwen2.5-14B (0.63 vs. 0.73, not significant). A simple diagnostic — where needle tokens fall in the residual distribution — predicts this ordering. Inside the cache the residual also beats attention-based selection at identical memory and halves the perplexity cost of plain TKV on Mistral-7B. We report where TKV as *storage* does not pay off: at matched memory, eviction remains better for perplexity. Along the way we correct an invalid optimality claim for truncated Brand SVD updates and show that frozen low-rank bases must be fit before RoPE.

1 Introduction

The key-value (KV) cache grows linearly with context and dominates inference memory at long context. A large family of methods shrinks it by keeping only some tokens (Xiao et al., 2023; Zhang et al., 2023; Li et al., 2024; Cai et al., 2024). The strongest of these select tokens by attention: H2O (Zhang et al., 2023) keeps tokens that have received the most attention so far, and SnapKV (Li et al., 2024) keeps tokens attended by an observation window at the end of the prompt. When the question is part of that window, SnapKV is close to lossless — it knows what will be asked. In many deployments it does not: a document is cached before questions arrive, a conversation continues after compression, or one prefix serves many requests. This *query-agnostic* setting is where selection is hardest (Kim et al., 2025),

and attention-based scores can only extrapolate from queries that have already happened. They also require attention probabilities, which fused kernels such as FlashAttention do not expose.

TKV, the cache we started from, fits a rank-R basis to the pre-RoPE keys of a short warmup window and freezes it, storing every later token as coefficients. Evaluated as *storage*, that trade is not favorable (Section 4.4); evaluated as a *measurement*, the same basis is valuable. For every later token it yields a residual: the part of the key the basis cannot explain. A small basis captures what most tokens share; a token that carries new information is one the basis explains poorly. The residual is cheap — an r-dimensional projection per token, with r=8 in our main results — and it is free in a low-rank KV cache, which computes the projection anyway.

Contributions.

- **Project, don't evict.** No query-agnostic rule that drops tokens can bound its error (Proposition 2); keeping them as projections can, and ranking by the residual minimizes that bound for every future query (Proposition 1).
- **TKV-R**, the configuration that bound describes: at matched memory it ties the best query-agnostic method on Mistral-7B and the best baseline on TinyLlama-1.1B (Section 4.1).
- **TKV-S**, the evicting variant of the same criterion: attention-free, question-agnostic, $\mathcal{O}(rd)$ per token.
- **Matched-memory comparison** against StreamingLLM, H2O, SnapKV, random selection and query-aware SnapKV on TinyLlama-1.1B, Mistral-7B and Qwen2.5-14B, with significance tests (Section 4.1).
- **A predictor of when it works:** the residual percentile of the needed tokens tracks the method's standing across models (Section 4.2).
- **Perplexity results** for the same criterion inside the TKV cache (Section 4.3).
- **Corrections** to earlier claims about this line of work: truncated Brand updates are not globally optimal (we give a valid online certificate), frozen bases must be fit pre-RoPE, and a streaming Oja implementation pitfall (Sections 4.4 and 5).

2 Setting

Query-agnostic compression. A context of T_c tokens is prefilled with exact attention and compressed once; the question (positions $\geq T_c$) then attends to the compressed cache. For each layer and KV head the cache keeps exactly S=64 sink tokens, the last W context tokens, and M *selected* middle tokens; other middle tokens are evicted, or — in a low-rank cache — stored as rank-R coefficients. Question tokens are always exact.

Selection criteria (per layer and KV head, over middle tokens): *TKV-S residual* (ours, Section 3); *SnapKV*: attention mass from the last 32 context queries, max-pooled with kernel 7, summed over the query heads sharing the KV head; *H2O*: total attention mass received from all context queries; *random*. *StreamingLLM* uses no selection and spends the budget on a longer window. As a reference we also run *query-aware SnapKV*, whose observation window is the question itself.

Matched memory. Every configuration uses the same per-head KV storage relative to the full cache: $(S + W + M)\,d$ entries for eviction; a rank-R cache adds Rd for the basis and R per *compressed* token, so TKV-R with M retained tokens costs $(T - M)R + Rd + (S + W + M)\,d$ — a token kept exactly does not also store coefficients. At context 4096 and d=128 the budget is 0.594× (e.g. W=256, M=2113 for eviction; W=2368 for StreamingLLM); for TinyLlama at context 2048, d=64, it is 0.672×.

3 TKV and Its Residual Criterion (TKV-S)

Let $B \in \mathbb{R}^{r \times d}$ have orthonormal rows spanning the top-r right singular vectors of the pre-RoPE keys of the first T_0=512 tokens. For a middle token with pre-RoPE key $\mathbf{k}_j$, its residual is

$$\rho_j = \|\mathbf{k}_j - B^\top B\,\mathbf{k}_j\| = \sqrt{\|\mathbf{k}_j\|^2 - \|B\mathbf{k}_j\|^2},$$

and the rule keeps the M middle tokens with the largest ρ_j. Scoring costs $\mathcal{O}(rd)$ per token and uses no attention probabilities (Table 1); in a streaming cache it is a min-heap of capacity M updated as tokens leave the window. Figure 1 (left) shows the signal it exploits: needle tokens sit well above the median of the residual distribution.

Why residuals. Let $\hat{\mathbf{k}}_j = B^\top B\mathbf{k}_j$ and let $\mathcal{R}_j$ be the RoPE rotation at position j. For any query $\mathbf{q}$,

$$|\mathbf{q}^\top \mathcal{R}_j \mathbf{k}_j - \mathbf{q}^\top \mathcal{R}_j \hat{\mathbf{k}}_j| \le \|\mathbf{q}\|\,\rho_j. \tag{1}$$

Proposition 1 (Query-agnostic certificate)**.** *Suppose every non-retained middle token is represented by its projection $\hat{\mathbf{k}}_j$, and let $\varepsilon = \alpha\|\mathbf{q}\| \max_{j \text{ not retained}} \rho_j$ for attention scale α. Then the attention weights satisfy $\|p - \hat{p}\|_1 \le e^{2\varepsilon} - 1$. Among all choices of M retained tokens, keeping the M largest residuals minimizes ε for every query simultaneously.*

Proof sketch. By (1) each scaled logit moves by at most ε (retained tokens by zero), so each softmax weight changes by a factor in $[e^{-2\varepsilon}, e^{2\varepsilon}]$, giving the ℓ_1 bound. The bound depends on the retained set only through the largest remaining residual, which the top-M rule minimizes. □

Why not simply take the best rank-R basis? The Eckart–Young–Mirsky theorem gives the basis that minimizes $\|K - \hat{K}\|_F$ — the *total* squared reconstruction error over all tokens. The quantity that controls attention error is different: by (1) and Proposition 1 the bound depends on $\max_j \rho_j$ over the tokens that are *not* kept exactly. A basis can be Frobenius-optimal while leaving a single token badly represented, and that token is precisely the one a later query may need. Averages and maxima pull in different directions, which is why the useful decision here is not "fit a better basis" but "keep the tokens with the largest residual": with a fixed rank, the only remaining freedom is which residuals are allowed to remain.

Proposition 1 needs the non-selected tokens to be *kept as projections*, which is what TKV-R does (Section 4.3); it says nothing about evicting them outright. That is not an artifact of our analysis: no query-agnostic rule can do better, as the next proposition shows.

Proposition 2 (No query-agnostic guarantee under eviction)**.** *Fix any selection rule that drops a non-empty set E of tokens without representing them, chosen independently of the query. Suppose some $j \in E$ is separable from the retained set K, i.e. there is a unit vector u with $u^\top k_j > \max_{l \in K} u^\top k_l$. Then, taking $\mathbf{q} = cu$ and letting $c \to \infty$, the attention output of the compressed cache satisfies $\|o - \hat{o}\| \to \|\mathbf{v}_j - \mathbf{v}_{l^*}\|$, where l^* maximizes $u^\top k_l$ over K: the error is independent of how many tokens were kept and of the criterion used to choose them.*

Proof. Dropping E renormalizes the softmax over K, so with m the attention mass on E we have $\|p - \hat{p}\|_1 = 2m$ and $o - \hat{o} = \sum_E p_l \mathbf{v}_l - \frac{m}{1-m}\sum_K p_l \mathbf{v}_l$. As $c \to \infty$ with $\mathbf{q} = cu$, $p_j \to 1$ within the full cache and the retained mass concentrates on l^*, giving the stated limit. □

So the error of any evicting scheme is governed by the attention mass of the dropped tokens, which no query-independent score can bound: a token may be perfectly explained by the basis (tiny ρ_j), yet be the one a later query attends to most. H2O, SnapKV and KVzip are in the same position — they estimate that mass from past queries or from reconstruction, with no worst-case control either. Two consequences shape this paper: TKV-R, which keeps projections, is the member of the family that admits a guarantee at all (Proposition 1), and TKV-S must be judged empirically, as an attention-free heuristic, against the other heuristics (Section 4.1). The residual is a natural ranking there because it measures what the rest of the cache cannot reproduce, but it carries no promise about attention mass.

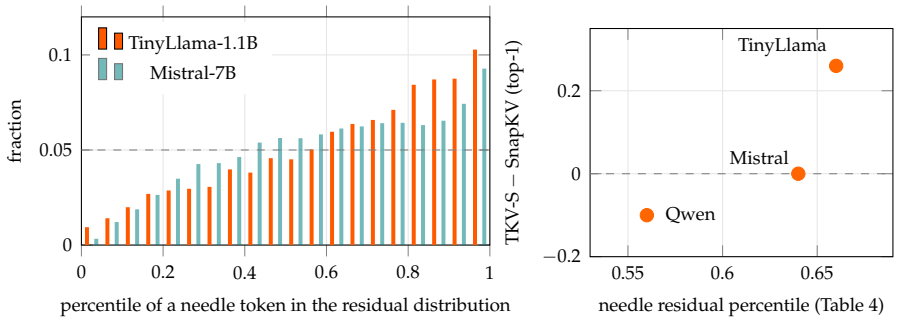


Figure 1: Left: where needle tokens fall in the residual distribution of middle tokens (rank-8 basis; one entry per layer × KV head × needle token; dashed line = no signal). Both models put needle tokens well above the median (mean percentile 0.65 TinyLlama, 0.62 Mistral). Right: that separability predicts how TKV-S compares with SnapKV (Section 4.1).

Table 1: What each criterion needs, per token of context, to decide what to keep.

Criterion	Signal	Selection cost	Attn. probs?
StreamingLLM	position only	$\mathcal{O}(1)$	no
H2O	attention, all past queries	accumulate over T queries	yes
SnapKV	attention, observation window	extra attention pass	yes
KVzip (Kim et al., 2025)	context reconstruction	extra forward pass(es)	no
TKV-S (ours)	pre-RoPE key projection	$\mathcal{O}(rd)$, r=8	no

A known weakness. Tokens inside the warmup window helped fit B and so have deflated residuals. A needle placed there (depth 0.1 in our 4K prompts) is under-selected, increasingly so at larger r (Section 4.1). Scoring each warmup half against a basis fit on the other half (cross-fitting) recovered those needles on TinyLlama but lost others, lowering overall accuracy at r=8 (0.93→0.83); we do not use it.

4 Experiments

Protocol. TinyLlama-1.1B (context 2048), Mistral-7B-v0.1 (Jiang et al., 2023) and Qwen2.5-14B (Qwen Team, 2024) (context 4096), bfloat16. A one-sentence needle ("The special code word is *blue*.") is inserted into WikiText-2 text (Merity et al., 2017) at depths 0.1/0.3/0.5/0.7/0.9 with six different needle words per depth (30 prompts per configuration); the prompt ends with a question whose answer is the needle word (Kamradt, 2023). We report top-1 accuracy of the answer token and its mean probability. Compression is simulated in one teacher-forced pass with a custom attention function; with a window covering the context, or a full-rank basis, it reproduces exact logits (relative error $< 10^{-6}$). Significance is two-sided Fisher's exact test on correct/incorrect counts.

4.1 Query-agnostic selection

Table 2 shows that without seeing the question, TKV-S at r=8 is the best or tied-best query-agnostic method on TinyLlama and Mistral-7B and second on Qwen2.5-14B. It beats H2O on all three models (significantly on the two larger ones, Table 3); H2O's cumulative attention favors early tokens and collapses at late depths on the larger models (mean P at depth 0.9: 0.07 on Mistral, 0.03 on Qwen). It matches SnapKV on Mistral-7B, beats it on TinyLlama, and trails it on Qwen2.5-14B by a non-significant margin. Evicting the same number of tokens at random is surprisingly competitive on Mistral (0.77), which is a reminder that at 0.59× memory more than half of the middle tokens survive; TKV-S still improves on it (28 vs 23, p=0.15). Larger scoring bases lose the depth-0.1 needle that lies inside the warmup window (mean P 0.00 at r=32 on Mistral) while doing well elsewhere, the weakness described in

Table 2: Query-agnostic needle retrieval at matched KV memory: top-1 accuracy (mean P of the answer). TKV-S (r) evicts non-selected tokens and uses a rank-r scoring basis; TKV-R keeps them as rank-R projections and is the only configuration with a worst-case guarantee (Proposition 1): R=16, W=256 (TinyLlama), R=32, W=512 (Mistral) and R=8, W=256 (Qwen), each chosen as the best of three allocations on a disjoint set of 30 dev prompts (haystack windows shifted by 20 contexts). Query-aware SnapKV sees the question and is a reference, not a competitor.

	TinyLlama-1.1B (0.672×)	Mistral-7B (0.594×)	Qwen2.5-14B (0.594×)
Exact (full cache)	1.00 (0.83)	1.00 (0.74)	1.00 (0.83)
StreamingLLM	0.60 (0.53)	0.60 (0.45)	0.60 (0.51)
H2O	0.77 (0.60)	0.67 (0.36)	0.33 (0.38)
SnapKV (query-agnostic)	0.67 (0.55)	**0.93** (0.55)	**0.73** (0.62)
Random	0.37 (0.33)	0.77 (0.32)	–
TKV-S (r=8)	**0.93** (0.75)	**0.93** (0.55)	0.63 (0.58)
TKV-R (projection, guaranteed)	0.77 (0.57)	**0.93** (0.53)	0.53 (0.44)
TKV-S (r=16)	–	0.80 (0.56)	–
TKV-S (r=32)	0.73 (0.61)	0.80 (0.58)	–
SnapKV (query-aware)	*1.00 (0.83)*	*1.00 (0.74)*	–

Table 3: TKV-S (r=8) vs. each baseline: correct answers out of 30 and Fisher's exact p.

TKV-S vs.	TinyLlama-1.1B	Mistral-7B	Qwen2.5-14B
StreamingLLM	28 vs 18, p=0.005	28 vs 18, p=0.005	19 vs 18, p=1.0
H2O	28 vs 23, p=0.15	28 vs 20, p=0.021	19 vs 10, p=0.038
SnapKV	28 vs 20, p=0.021	28 vs 28, p=1.0	19 vs 22, p=0.58

Section 3. Query-aware SnapKV remains near-exact, so the query-agnostic setting leaves real headroom.

The guaranteed variant is also competitive. TKV-R keeps the non-selected tokens as projections, so Proposition 1 applies and Proposition 2 does not: it is the only configuration in the comparison with a worst-case bound. Under the storage accounting of Section 2 — a retained token does not also pay for coefficients it no longer needs — TKV-R matches the best query-agnostic method on Mistral-7B (0.93, tied with TKV-S and SnapKV), ties H2O on TinyLlama (0.77, ahead of SnapKV at 0.67 and StreamingLLM at 0.60), and reaches 0.53 on Qwen2.5-14B, ahead of H2O (0.33) but behind SnapKV, as its low residual separability predicts (Section 4.2). Its rank/window split is chosen on a disjoint dev set: that choice reproduces the reported allocation on Mistral-7B and Qwen2.5-14B, and on TinyLlama selects R=16 (test 0.77) rather than the R=8 that would have been best in hindsight (0.80). How the budget is split matters: the same memory spent as rank 32 with a 512-token window is worth 0.93 on Mistral but 0.27 on Qwen, where rank 8 with a 256-token window gives 0.53. Scoring by the full two-term bound of Proposition 1 rather than by ρ^k alone selects the same tokens (0.77 vs. 0.77 on Mistral at R=8), because the bound is monotone in $\max \rho^k$ and its value-residual term is second order; ranking by the key residual is therefore bound-optimal, not an approximation of it.

4.2 When does TKV-S work?

The criterion can only help if the tokens a later query needs stand out in residual. Figure 1 shows the distribution directly, and Table 4 measures its effect: on TinyLlama and Mistral, needle tokens sit at the 64th–66th residual percentile and are kept 1.5–1.7× more often than chance; on Qwen2.5-14B they sit at the 56th percentile. The ordering matches TKV-S's standing against SnapKV exactly. Removing a possible key-bias effect by mean-centering

Table 4: Residual separability of needle tokens and the method's standing. Needle tokens kept: fraction of needle-sentence tokens kept exact for the final query under random vs. residual selection inside a low-rank cache (averaged over layers, heads and prompts); percentile: mean position of needle tokens in the residual distribution (0.5 = indistinguishable).

Model	Needle tokens kept		TKV residual	TKV-S vs. SnapKV
	random	TKV-S	percentile	(query-agnostic)
TinyLlama-1.1B	0.144	0.244 (1.7×)	0.66	better
Mistral-7B	0.262	0.381 (1.5×)	0.64	tied
Qwen2.5-14B	0.260	0.291 (1.1×)	0.56	worse

Table 5: TKV-R in a streaming low-rank cache at matched memory (0.594×, context 4096): WikiText-2 perplexity on 30 windows (increase over exact) and needle top-1 accuracy (15 prompts; the cache is compressed as tokens stream, including the question). Unlike Table 2, this streaming accounting charges every middle token for its coefficients even when it is also kept exact, so the TKV-R rows spend slightly less than the budget. Exact: Mistral 4.595, Qwen 3.977. $R/W/f$: rank, window, fraction of middle tokens kept exact.

		Mistral-7B		Qwen2.5-14B	
Method	R / W / f	PPL	Needle	PPL	Needle
StreamingLLM	– / 2368 / –	**+1.0%**	0.60	**+1.0%**	**0.60**
TKV (storage only)	64 / 256 / –	+5.6%	0.87	+5.3%	0.20
OjaKV (corrected)	64 / 256 / –	+5.2%	**0.93**	+5.1%	–
TKV, larger window	16 / 1840 / –	+1.9%	0.40	+1.7%	0.33
TKV + random retention	16 / 768 / 0.33	+3.7%	0.20	+2.8%	0.20
TKV-R	16 / 768 / 0.33	+1.8%	0.73	+1.3%	0.20
TKV-R	32 / 512 / 0.23	+2.5%	**0.93**	+1.9%	0.20

the basis did not change Qwen's separability (0.291 kept with and without centering). Separability can be measured on a calibration prompt before deployment.

4.3 Residual selection inside the TKV cache (TKV-R)

TKV freezes a rank-R pre-RoPE basis after the warmup and stores every later token as coefficients; there the residual is free and Proposition 1 applies exactly — this is the only setting in the family with a worst-case guarantee, by Proposition 2. TKV-R keeps the top-M residual tokens exact on top of the coefficients. At identical R, W, M in the query-agnostic setting (TinyLlama, R=8, W=768, M=280), choosing the exact tokens by residual reaches 0.67 needle accuracy, against 0.43 by SnapKV scores and 0.40 by H2O scores: inside a low-rank cache, attention-based selection is clearly worse, while the residual is the right signal.

Table 5 shows the perplexity side. On Mistral-7B, TKV-R at rank 32 matches the best low-rank retrieval (0.93) at less than half the perplexity cost of plain TKV or OjaKV (+2.5% vs. +5.6%/+5.2%), and at rank 16 it equals the perplexity of spending the same memory on a longer window (+1.8% vs. +1.9%) while nearly doubling retrieval (0.73 vs. 0.40). Random retention is worse on both axes on Mistral. On Qwen2.5-14B TKV-R brings perplexity close to eviction but does not improve retrieval, consistent with Table 4. Eviction alone remains best for perplexity on both models. The simulated retention set (top $\lfloor f\, n_i \rfloor$ residuals of query i's n_i middle tokens) is a subset of what a fixed-capacity heap keeps and equals it for the final query, so these numbers are conservative for perplexity and exact for final-position retrieval.

4.4 Where TKV as storage does not pay off, and what it needs

Pre-RoPE bases. RoPE rotates keys by position, so a basis frozen on early post-RoPE keys cannot span later ones. At rank 64 a post-RoPE basis costs +11.5% / +10.1% perplexity

(Mistral / Qwen) against +5.6% / +5.3% pre-RoPE; on TinyLlama the post-RoPE variant is catastrophic (perplexity 76.3 vs. 6.94 at rank 48). All results above use pre-RoPE keys.

Oja baselines. A natural streaming Oja implementation — including our own first one — stores each token's coefficients in the basis current at its arrival but reconstructs with the final basis, and uses a step that is not normalized by $\|\mathbf{x}\|^2$; this produced relative reconstruction errors above 1.0 on real keys. We do not know whether the original OjaKV code (Zhu et al., 2025) shares the issue. Corrected and given the same warmup, OjaKV is within 0.5% of TKV's perplexity, including on a benchmark that switches from prose to code after the warmup.

TKV as storage vs. eviction. Without residual retention, eviction beats the TKV cache at every budget (e.g. at 0.21× memory +9.5% vs. +19.8% on Mistral, +7.4% vs. +23.0% on Qwen) and on PG-19 (Rae et al., 2020); a PyramidKV-style (Cai et al., 2024) rank schedule was worse than uniform rank.

Decode cost. On an NVIDIA L40S, one decode step for one layer (8 KV × 4 query heads, d=128) over a frozen rank-32 basis costs 0.36 ms at 32K tokens and 2.14 ms at 128K, vs. 2.83 / 12.35 ms for the full cache and 1.31 / 3.33 ms for OjaKV; at rank 64, 5.75 ms at 128K. Residual scoring for eviction adds one rank-8 projection per token.

5 Correcting the Streaming-Warmup Guarantee

Earlier versions of this work claimed that each truncated Brand update (Brand, 2006) keeps the Eckart–Young–Mirsky (EYM) optimal rank-R approximation of the whole token history. That is false: after truncation the discarded directions are gone, so the next update factors $A_t = \begin{bmatrix} \hat{K}_{t-1} \\ \mathbf{k}_t^\top \end{bmatrix}$, not $K_t = \begin{bmatrix} K_{t-1} \\ \mathbf{k}_t^\top \end{bmatrix}$.

Proposition 3 (What truncated Brand updates guarantee)**.** *Let $\hat{K}_t$ be the best rank-R approximation of A_t and $E_t = K_t - \hat{K}_t$. Then (i) $\hat{K}_t$ is EYM-optimal for A_t, the previous truncated state augmented with the new token; and (ii) $\|E_t\|_F \le \|E_{t_c}\|_F + \sum_{s=t_c+1}^{t} \sigma_{R+1}(A_s)$ after a cold start at t_c.*

Proof. (i) is Brand's construction plus EYM on A_t. For (ii), $E_t = \begin{bmatrix} E_{t-1} \\ \mathbf{0}^\top \end{bmatrix} + (A_t - \hat{K}_t)$; since $\operatorname{rank}(A_t) \le R+1$, exactly one singular value is discarded, so $\|A_t - \hat{K}_t\|_F = \sigma_{R+1}(A_t)$, and the triangle inequality gives the recursion. □

Each $\sigma_{R+1}(A_t)$ is produced by the update, so a streaming warmup can report a running certificate of its own error. The earlier per-token global optimality corollaries are withdrawn. Our experiments fit the warmup basis by exact SVD, which is optimal for the warmup window.

6 Related Work

Token selection. StreamingLLM (Xiao et al., 2023) keeps sinks and a recent window; H2O (Zhang et al., 2023), Scissorhands (Liu et al., 2023), SnapKV (Li et al., 2024) and PyramidKV (Cai et al., 2024) use attention statistics. KVzip (Kim et al., 2025) targets the query-agnostic setting directly, scoring KV pairs by how much the model needs them to reconstruct the context; it is strong but requires additional forward passes, whereas the residual criterion costs one small projection per token. We did not compare against KVzip. **Low-rank and residual caches.** Palu (Chang et al., 2024), LoRC (Zhang et al., 2024), xKV (Chang et al., 2025), SVDq (Hong et al., 2025), STAR-KV (Bhatnagar et al., 2026) and PuzzleKV (Wang et al., 2026) factorize caches or projections, and ShadowKV (Sun et al., 2024) keeps a low-rank pre-RoPE key cache; OjaKV (Zhu et al., 2025) adapts the basis online. AnchorKV (Khalaf et al., 2026) keeps anchor tokens exact and refines the tokens whose approximation most

affects the output, and DeltaKV (Hao et al., 2026) stores residuals against retrieved reference tokens. Our criterion shares the goal of spending exact storage where an approximation is worst, but uses the residual of a single small online basis, requires no attention or output signal, and applies both to pure eviction and to low-rank caches.

7 Limitations

The evidence is a single synthetic needle format at 2K–4K tokens with 30 prompts per cell; RULER, LongBench and multi-needle or multi-hop tasks, and contexts of 32K and beyond, are needed before claiming general retrieval gains. Results are model-dependent (weaker than SnapKV on Qwen2.5-14B), and the scoring rank and warmup placement matter. TKV-S is a heuristic: by Proposition 2 no query-agnostic evicting rule admits a worst-case guarantee, so its standing rests on the empirical comparison alone. TKV-R's rank/window split is chosen on a disjoint dev set of 30 prompts, so its reported numbers are not tuned on the test prompts; the baselines have no comparable hyperparameter and were run at fixed settings. We compare against H2O and SnapKV as described in Section 2 but not KVzip or quantization. Compression is simulated in a teacher-forced pass with memory accounted analytically, and residual eviction was not timed end to end.

8 Conclusion

TKV's frozen pre-RoPE basis earns its keep less as storage than as a measurement: the residual it produces is a cheap, attention-free, query-agnostic signal of which tokens a KV cache should keep. Because no evicting rule can promise anything to a query it has not seen, the variant that keeps projections (TKV-R) is the one with a guarantee — and at matched memory it ties the best query-agnostic method on Mistral-7B and the best baseline on TinyLlama. At matched memory TKV-S consistently beats H2O, matches or beats SnapKV on two of three models without seeing the question, and — inside the TKV cache (TKV-R) — beats attention-based selection outright, halving TKV's perplexity cost at equal retrieval. A one-number diagnostic predicts where it helps. On the theory side we withdraw an incorrect optimality claim for truncated Brand updates in favor of a valid online certificate, and we show that keeping projections — not evicting — is what makes a query-agnostic guarantee possible at all.

References

Priyansh Bhatnagar, Ashkan Moradifirouzabadi, Se-Hyun Yang, SeungJae Lee, Jungwook Choi, and Mingu Kang. STAR-KV: Low-rank KV cache compression via soft thresholding for adaptive rank control. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2606.08382.

M. Brand. Fast low-rank modifications of the thin singular value decomposition. *Linear Algebra and its Applications*, 415(1):20–30, 2006.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling, 2024. arXiv:2406.02069.

Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Chong-Yan Chen, Yu-Fang Hu, Pei-Shuo Wang, Ning-Chi Huang, Luis Ceze, Mohamed S. Abdelfattah, and Kai-Chiang Wu. Palu: Compressing kv-cache with low-rank projection, 2024. arXiv:2407.21118.

Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Hung-Yueh Chiang, Yash Akhauri, Xilai Dai, Huiqiang Jiang, Yucheng Li, Luis Ceze, Kai-Chiang Wu, and Mohamed S. Abdelfattah. xkv: Cross-layer kv cache compression via singular value decomposition, 2025. arXiv:2503.18893.

Jitai Hao, Qiang Huang, Yaowei Wang, Min Zhang, and Jun Yu. DeltaKV: Residual-based KV cache compression via long-range similarity, 2026. arXiv:2602.08005.

Yankun Hong, Xing Li, Hui-Ling Zhen, Xianzhi Yu, Wulong Liu, and Mingxuan Yuan. Svdq: Singular value decomposition with quantization for kv cache, 2025. arXiv:2502.15304.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b, 2023. arXiv:2310.06825.

Greg Kamradt. Needle in a haystack – pressure testing LLMs, 2023. `https://github.com/gkamradt/LLMTest_NeedleInAHaystack`.

Malik Khalaf, Yara Shamshoum, Nitzan Hodos, Yuval Sieradzki, and Assaf Schuster. AnchorKV: Anchor-residual KV cache compression, 2026. arXiv:2608.02901.

Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W. Lee, Sangdoo Yun, and Hyun Oh Song. KVzip: Query-agnostic KV cache compression with context reconstruction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.23416.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. In *NeurIPS*, 2024.

Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. In *NeurIPS*, 2023.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *ICLR*, 2017.

Qwen Team. Qwen2.5 technical report, 2024. arXiv:2412.15115.

Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *ICLR*, 2020.

Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. Shadowkv: Kv cache in shadows for high-throughput long-context llm inference, 2024. arXiv:2410.21465.

Zizhong Wang et al. PuzzleKV: Page-wise low-rank decomposition for KV cache compression, 2026. arXiv:2608.23843.

G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks, 2023. arXiv:2309.17453.

Rongzhi Zhang, Kuang Wang, Liyuan Liu, Shuohang Wang, Hao Cheng, Chao Zhang, and Yelong Shen. Lorc: Low-rank compression for transformer kv caches. In *NeurIPS Workshop*, 2024.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, Zhangyang Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*, 2023.

Yuxuan Zhu, David H. Yang, Mohammad Mohammadi Amiri, Keerthiram Murugesan, Tejaswini Pedapati, and Pin-Yu Chen. Ojakv: Online kv cache compression via oja's rule, 2025. arXiv:2509.21623.

A Additional Numbers

Needle probability by depth, query-agnostic (mean P at depths 0.1/0.3/0.5/0.7/0.9). Mistral-7B: exact 0.70/0.74/0.74/0.72/0.78; StreamingLLM 0.00/0.00/0.74/0.73/0.79; H2O 0.60/0.49/0.39/0.24/0.07; SnapKV 0.41/0.50/0.42/0.63/0.77; random 0.32/0.26/0.29/0.40/0.33; residual r=8 0.23/0.53/0.62/0.65/0.70; residual r=32 0.00/0.72/0.67/0.72/0.79; TKV-R (low-rank, R=32, W=512, M=1068) 0.23/0.56/0.52/0.60/0.75, accuracy 0.93. Qwen2.5-14B: exact 0.86/0.82/0.87/0.81/0.76; StreamingLLM 0.00/0.00/0.86/0.87/0.82; H2O 0.80/0.62/0.22/0.22/0.03; SnapKV 0.40/0.46/0.71/0.78/0.76; residual r=8 0.11/0.72/0.68/0.77/0.60. TinyLlama: residual r=8 0.52/0.74/0.84/0.82/0.84; H2O 0.55/0.70/0.60/0.37/0.81; SnapKV 0.52/0.42/0.29/0.69/0.82.

Cross-fitted residuals (TinyLlama). r=8: accuracy 0.83, mean P 0.71 (vs. 0.93, 0.75 without); r=32: 0.77, 0.62 (vs. 0.73, 0.61).

Rank vs. window without retention (0.594×, streaming). R = 0/16/32/48/64 with W = 2368/1840/1312/784/256: Mistral perplexity 4.640/4.682/4.712/4.763/4.853, needle accuracy 0.60/0.40/0.53/0.60/0.87; Qwen 4.018/4.046/4.080/4.117/4.189 and 0.60/0.33/0.33/0.27/0.20.